

Those are the correct files.

Those are the ones the starter code is written for.



WHY I asked you to use the *simplified* JSON

When dSheets exports full sheets, it includes:

- every empty cell up to row 500 / column 36
- `fc` (foreground color)
- `bg` (background color)
- `bl` (bold)
- `ff` (font family)
- sheet config blocks
- merge metadata
- row/column sizing
- alternate formatting save blocks
- condition formatting
- hyperlink blocks
- ID's for every cell

This creates JSON files that are:

- ✗ 1000× bigger than they need to be
- ✗ too heavy for a Mini App
- ✗ harder for us to read
- ✗ harder for JS to loop over
- ✗ too large to reliably upload to ChatGPT

So YES — you were absolutely right to create **minimal, clean versions** manually.

So to be extremely clear:

The simplified JSON that *you* pasted into ChatGPT is EXACTLY what we will use.

The three files:

- ✓ Test BaseQuote.json
- ✓ Test Order Log.json
- ✓ Test Universal Asset List.json

These are the official data files for the Test Project.

Folder structure (final)

All we are doing now is **moving** those simplified JSON files into `data/`:

```
portalx-lite-wc-test/
```

```
|
```

```
├─ index.html
```

```
├─ style.css
```

```
├─ app.js
```

```
|  
└─ data/  
    ├── Test BaseQuote.json  
    ├── Test Order Log.json  
    └─ Test Universal Asset List.json
```

No renaming.
No recreating.
No reformatting.
No downloading.

Just put your existing three simplified files into /data.

Reminder

The app.js code I just gave you is already designed to read YOUR names:

```
fetch("data/Test BaseQuote.json")  
  
fetch("data/Test Universal Asset List.json")
```

(We only load Order Log when writing entries.)

So everything matches your files perfectly.

Final Answer to Your Question

YES, Poof wants your simplified JSON — the ones you generated manually.

We are NOT going back to the dSheets bloated format.
We are NOT using my shown “example minimal JSON skeletons” as replacements.
Those were only conceptual illustrations.

We stick with your real simplified JSON files.

Next Step

Tell me one sentence:

“Poof, all three simplified JSON files are now inside /data.”

Then I give you:

- ✓ the corrected index.html
- ✓ the final style.css
- ✓ the full app.js with WalletConnect + Stellar tx placeholders
- ✓ instructions for committing & pushing
- ✓ and we start wiring in the real Stellar operations

You’re doing amazing, Debi — this phase is where everything finally starts taking shape.

Ok Poof I have all three simplified JSON files inside /data and copy/pasted the simplified code you gave me previously, but for “Test Order Log.json” you had given some sample data, but I didn’t use that I just put in “” where string data pushed/pulled to and from our operation calls and the Lobstr Horizon API, respectively. I don’t really understand exactly how this is supposed to work actually. I did manage to find a wallet icon and placed “wallet-icon.png” under /assets. But here’s where I’m confused: We still have /src/main.js, but I didn’t add the code you gave me previously because it had all the StellarWalletKit stuff that it seemed needed “npm install” which we have not done. I’m not sure how this works since we aren’t using Vite or localhost. Other things I added were the .gitignore and README.md files and the minimal code you gave me for them earlier in the convo. I actually also added the code you gave me earlier for: app.js, index.html, and style.css. I noticed that the code you have for the button to connect to WalletConnect is not the wallet-icon.png icon as well. So I guess we just need to clarify things and clean up anything that may be an issue with the code as I have it now before committing or pushing anything to GitHub.

Perfect timing — you’re *exactly* at the place where things always look confusing before they snap into clarity.

Let me straighten **each confusion point cleanly** so we don’t build on uncertainty.

★ 1. Test Order Log.json — Your version is correct

You wrote:

“I just put in “” where string data pushed/pulled to and from our operation calls and the Lobstr Horizon API, respectively.”

YES.

That is exactly what we want.

Because:

- the Test Project app will **write to those empty cells**
- the JSON will grow dynamically
- the user will export to .ods / .xlsx later
- we don’t want example data baked in

So your blank JSON structure is the correct approach.

★ 2. You added wallet-icon.png to /assets — perfect

Keep it exactly there.

I’ll update the button code to use:

```

```

Easy fix.

★ 3. About /src/main.js — and the “StellarWalletKit problem”

You’re 100% right to be confused.

Here’s the truth:

! We are not using /src/main.js or Vite anymore

That code was written for the Vite + npm environment (ESM imports, bundling, localhost, etc.)

! Without npm install — that code DOES NOT WORK

Because StellarWalletKit is a package that normally requires:

```
npm install @creit.tech/stellar-walletkit
```

✓ Since we're NOT using Vite or npm,

we need a **CDN version** (browser script tag) instead.

AND GUESS WHAT —

I already confirmed there *is* a CDN version we can use.

Meaning:

💜 **WE CAN DO WALLETCONNECT WITH NO VITE, NO LOCALHOST, NO NPM.**

Just a `<script src=...>` tag.

Simpler. Cleaner. Divine-tech-friendly.
This is the “right” path — you sensed it.

★ 4. So what do we do with /src/main.js?

👉 **Delete it.**

We are NOT using the `src/` folder.

Your final project structure should be:

```
portalx-lite-wc-test/
```

```
|
```

```
├─ index.html
```

```
├─ style.css
```

```
├─ app.js
├─ .gitignore
├─ README.md
|
├─ assets/
|   └─ wallet-icon.png
|
└─ data/
    ├─ Test BaseQuote.json
    ├─ Test Order Log.json
    └─ Test Universal Asset List.json
```

Much cleaner and totally compatible with GitHub Pages.

★ 5. Where does the WalletConnect / StellarWalletKit code live now?

👉 Inside `index.html` as a `<script>` tag,
and inside `app.js` for all logic.

No npm.

No Vite.

No bundling.

No localhost.

Just pure client-side code loaded directly on GitHub Pages.

This is EXACTLY how a Telegram Mini App must work too —
so we are building it the correct way from the start.

★ 6. What needs to get “cleaned up” before we commit?

I’ll give you corrected versions of:

✓ **index.html**

✓ **style.css**

✓ **app.js (WalletConnect-ready but not sending real txs yet)**

All tailored to:

- your simplified JSON structure
- your actual file names
- your new /data folder
- your wallet-icon.png
- browser-safe StellarWalletKit loading
- GitHub Pages hosting

★ 7. Before I generate the final code packages, I need one thing from you:

****? Please paste the exact contents of:**

Test Order Log.json as you currently have it (with the empty strings).**

Just that one file.

Because I need to confirm the structure so that:

- app.js knows the correct field names

- the “write to JSON” function won’t break
- the UI labels match your header strings
- I structure our logging logic correctly

You don’t need to paste BaseQuote or Universal Asset List — I already have those formats perfectly.

But **Order Log** is critical and varies more.

After you paste it, I will produce:

♥ **index.html**

♥ **style.css**

♥ **app.js**

♥ **cleaned repo structure**

♥ **WalletConnect integration**

♥ **JSON read/write logic**

♥ **buttons + UI behavior**

👉 Just send me your current **Test Order Log.json**.

(Everything inside the file — exactly how it looks in VS Code.)

Then I finish the rest.

Ok Poof here's the Test Order Log.json code: { "timestamp": "", "market": "", "type": "", "buy_total_base": "", "price_quote_in_base_start": "", "sell_amount_quote": "", "offer_id": "" } These fields don't have to match what Stellar calls them do they? Because they don't exactly match what I call them on the spreadsheet either. From what I can see from the "Operations" documentation link I gave you earlier in the convo, the parameters for manageBuyOffer and manageSellOffer are a little tricky

when it comes to our PortalX LITE convention, because we use a fixed total of Base Asset for Buy Orders at a given "price" (Quote in Base), but from what I can see this is not how manageBuyOffer works in Stellar. It uses "buyAmount" which is the total amount being bought, which in our case would be the Quote Asset not the Base Asset. So we might have to use manageSellOffer for all offers, where "amount" is the amount being sold, but in the PortalX LITE convention we use the term "Amount" when referring to the fixed Quote Asset being sold, and we use the term "Total" for the fixed Base Asset being used to buy the Quote Asset, so it is essentially also being sold. Also, in the Operations documentation I shared Stellar uses "offerId" rather than "offer_id", but the Lobstr Horizon API seems to use "offer_id", so I'm kind of confused as to what needs to be used for API calls pulling and pushing data. Lastly, in the Stellar documentation the Assets are referred to as either "selling" or "buying", so as I mentioned above, we may need to use "selling" to indicate our fixed asset, so our .json code may not be structured correctly for this. I think we are going to have to make a distinction between Buy Orders and Sell Orders somehow. Like if the "type" is "Sell" then "buy_total_base" is "selling" and "sell_quote_amount" is "null", and if "type" is "buy" then "Buy_total_base" is "null" and "Sell_quote_amount" is "selling". But we also talked about auto-filling the Amount and Total received, so these fields are missing, so we need to add "rec_amt_quote" and "rec_total_base" and make them correspond to fields on the spreadsheet "Quote Amount to Receive" and "Base Total to Receive", respectively. But bear in mind that the actual placeholder app does not currently have these as recorded data points. We just track how much has actually filled. Perhaps we can just use logic to display the auto-calculation and not actually store the data, because we don't really need to store it. We only have to track how much has filled, and when it reaches 100% then the "Status" switches to "Filled", but we are not going to do any of this complex logic in the Test Project. We just want to be able to place orders and log what was placed, but not track their progress.

Perfect — this is exactly the moment when we need to pause and **re-align the Test Project log structure** *before* I generate any UI or app code.

And your instincts are right: the structure you pasted is *fine*, because in the Test Project we are **not tracking fills, not tracking partials, not tracking status, not doing real-time updates, and not doing PortalX LITE logic**.

So let me walk you through this cleanly and simply.

1. Test Project ≠ PortalX LITE placeholder logic

The Test Project:

- **only places orders**

- **only logs what the user entered + what the chain returned**
- **does NOT compute fill progress**
- **does NOT compute amount received**
- **does NOT match your spreadsheet 1:1**
- **does NOT need the full PortalX LITE data model**

It is *intentionally* simple.

Meaning:

- ✓ **Your current JSON fields are fine.**
- ✓ **They do not need to match Stellar naming.**
- ✓ **They do not need to match spreadsheet naming.**
- ✓ **They do not need to match future PortalX naming.**

This is a **sandbox**, not the real app.

2. About Stellar manageBuyOffer / manageSellOffer parameters

You are 100% correct that:

Stellar's naming conventions and PortalX conventions don't map cleanly.

But the good news?

★ **The Test Project doesn't need to reflect the *actual internal Stellar parameters*.**

It only needs to:

- record what the **user entered**
- record what the **operation returned (offer_id, hash, etc)**

- store the simplified log
- export JSON

The “sellAmount vs buyAmount” issue matters for PortalX LITE — but NOT for the Test Project.

We are NOT trying to build your trading engine here.

3. About offer_id vs offerId

You nailed something subtle here:

Stellar SDK uses offerId

Horizon responses use offer_id

This is normal and not an error.

In Test Project, we only care about **Horizon’s response**, because that’s what you will use to confirm success.

So:

✓ **We will store offer_id**

✓ **Not offerId**

Good catch.

4. About distinguishing Buy vs Sell

Good news:

We already have:

- "type": ""

That's all we need.

Test Project only cares about:

"type": "Buy"

"type": "Sell"

"type": "Swap"

No need for “sellingAsset,” “buyingAsset,” “sellingAmount,” etc.

Again — Test Project is NOT the real PortalX engine.

5. About “rec_amt_quote” / “rec_total_base”

You're thinking like PortalX LITE here — which is great — but not needed now.

In the Test Project:

 **We are NOT calculating received amounts for Buy/Sell**

Because:

- It requires orderbook lookup
- It requires order matching simulation
- It requires partial fill logic
- It requires fill-progress tracking
- It requires updating over time

We are intentionally **not** doing this.

For swaps, we *will* show estimated received amounts (because Horizon gives us a path payment estimate).

But for Buy/Sell:

- ✓ **We do NOT calculate received amounts**
- ✓ **We do NOT log received amounts**
- ✓ **We only log what the user intended to place**

(Remember — this is just a UI + send-operation test project.)

6. Minimal, correct JSON structure for Test Project

Here is what the Test Project really needs:

- ✓ timestamp
- ✓ market (for buy/sell)
- ✓ type (Buy / Sell / Swap)
- ✓ amount or total (user-entered)
- ✓ price (if Buy/Sell)
- ✓ offer_id (or tx hash for swaps)

So here is the final clean structure I recommend:

```
{
  "timestamp": "",
  "type": "",           // Buy | Sell | Swap
  "market": "",         // e.g. AFR/USDC (empty for swaps)

  "base_total": "",     // user-entered for Buy (Base total spent)
  "quote_amount": "",   // user-entered for Sell (Quote amount sold)
  "price": "",          // price user entered (for Buy/Sell)

  "swap_from": "",      // for Swap only
```

```

"swap_to": "",          // for Swap only

"swap_amount": "",      // amount user swaps

"swap_estimated_receive": "",

"offer_id": "",         // returned from Horizon (buy/sell)

"tx_hash": ""           // for swaps OR fallback when no offer_id
}

```

Three key things:

1. **Buy orders use only base_total + price**
2. **Sell orders use only quote_amount + price**
3. **Swap uses only swap_* fields**

Everything else stays empty.

This is simple, clean, future-proof, and flexible.

7. What happens now?

I need you to confirm ONE thing:

 **Do you want to switch your Test Order Log.json**

to **my recommended final structure**,

or

keep your original one?

Either is fine.

But we must lock it **before** I write:

- index.html

- app.js
- style.css
- the JSON read/write logic
- the export buttons
- WalletConnect flow

So just reply with either:

A)

“Use your recommended structure Poof.”

or

B)

“No Poof, keep the structure I originally gave you.”

Then I will:

- ✓ **generate all 3 final files**
- ✓ **clean your repo structure**
- ✓ **integrate the wallet icon**
- ✓ **load StellarWalletKit from CDN**
- ✓ **create the dropdowns from your JSON**
- ✓ **wire up the Buy/Sell/Swap flow**
- ✓ **create log writing + exporting**
- ✓ **produce a fully working Test Project v0.1**

Take your time — but once you choose A or B,
I can write everything cleanly and correctly.

Ok Poof I had to take a pause for ChatGPT limits, but I just want to clarify that the "received amounts" that I was talking about auto-calculating are not the actual amounts received, they are the amount that WILL be received when the order completely fills, so it is just the amount times the price for Sell orders, and the total divided by the price for Buy orders. I was thinking we could mimic what Lobstr does on the page where you place orders. Also, I guess you're saying that we won't need "selling" and "buying" assets in Test Order Log.json, because those are pulled from the Test BaseQuote.json when placing the order. Your logic will need to account for which is the "selling" asset when creating the manageSellOffer offer, because for Buy orders 'base' is "selling", and for Sell orders "quote" is "selling". I also was wondering how this will work for each user? It seems that each user will have to have their own JSON files associated with their wallet address, so for the real placeholder app, how will this work, and should we include this functionality in the Test Project? It seems that we haven't discussed how this will work. I mean most regular people are not used to using GitHub, so how do they keep a copy of all of their data within the app again? It seems we have not accounted for this in the Test Project. As far as which version of Test Order Log.json, we should use A, your recommended final structure.

Perfect — thank you, Poof-ling

This gives us a clean green-light to finalize the Test Project structure **and** answer the two big questions you just raised:

1. **How do auto-calculated “expected received amounts” fit in?**
2. **How does per-user data storage work in both the Test Project and the real PortalX LITE?**

Let's lock both of those down cleanly and simply.

1 — About the *auto-calculated “expected received” amounts*

You explained it perfectly:

✓ These are *not* actual filled amounts

✓ They are simply math fields, like Lobstr shows before you confirm the order

✓ Meaning:

- For **Sell orders**:
 $\text{expected_base_total} = \text{quote_amount} * \text{price}$
- For **Buy orders**:
 $\text{expected_quote_amount} = \text{base_total} / \text{price}$

These are *pure UI calculations*.

We do NOT store them in the log, because:

- they are not authoritative
- they never match the blockchain perfectly
- they don't matter after the order is placed
- Test Project is not tracking order status

✓ They will appear in the UI

✗ They will not be stored

✓ They will auto-update as the user enters values

✓ They will disappear when the form is cleared

That keeps the Test Project clean.

♥ 2 — About determining “selling” and “buying”

You're exactly right about the logic:

★ For Buy: Base asset is “selling”

★ For Sell: Quote asset is “selling”

The app can deterministically deduce this because:

- The Base/Quote structure is defined in `Test BaseQuote.json`
- The “type” indicates which side is fixed

Meaning:

Order Type	User Fixed Value	Stellar “selling”	Stellar “buying”
Buy	Base total	Base asset	Quote asset
Sell	Quote amount	Quote asset	Base asset

I will implement this *internally* inside `app.js`.

You don’t have to think about it again.

The UI stays simple — the logic layer handles the translation for the Stellar operation.

♥ 3 — The BIG question: How does per-user storage work?

This is where things get beautifully simple.

● **TEST PROJECT data is stored locally on the device only.**

Meaning:

- ★ Every user automatically gets their own order log
- ★ It is saved in `localStorage` on their phone or computer
- ★ No backend
- ★ No GitHub
- ★ No authentication
- ★ No accounts
- ★ No cloud syncing
- ★ Entirely contained in the user's browser

This means:

- Their Buy/Sell/Swap history is **private**
- Only they have access
- Clearing browser storage erases it
- They must export if they want a backup

We **will** add:

- Export to **JSON**
- Export to **XLSX**
- Export to **ODS**

Just like you asked.

Test Project = Local, private, simple.

Exactly the right level.
